COMP90041
Programming and Software Development

Lecture 3 - Classes and Methods - I

Amani Abusafia

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

THE UNIVERSITY OF MELBOURNE

Classes and Methods - Overview

In this part of the lecture, you will learn about:

- **Class definitions**
  - Class structure
  - Variables
  - Methods
- **Constructors**

Why object-oriented programming?

- Represent the real world
- Imagine you are working for a car rental company that needs to keep track of its cars.

Car

Characteristics

- Manufacturer(e.g., Toyota, Tesla).
- Model (e.g., Camry, Model S).
- Year built (e.g., 2022).
- Is rented (e.g., true, false).
- Rental Price (e.g. per day, 5).

Actions

- Calculate Rental price

Why object-oriented programming?

- Represent the real world
- Imagine you are working for a car rental company that needs to keep track of its cars.

Car

Characteristics

```
String manufacturer;
String model;
int yearBuilt;
boolean isRented;
float rentalPrice;
```

Actions

```
float price = days * rentalPrice;
```

Why object-oriented programming?

Car1

Characteristics

```
String manufacturer;
String model;
```

Actions

```
float price = days * rentalPrice;
```

What if we have 1000 cars?

```
String model2;
int yearBuilt2;
boolean isRented2;
float rentalPrice2;
```

A better Way to organize…

Characteristics

```
String manufacturer;
String model;
int yearBuilt;
boolean isRented;
float rentalPrice;
```

Actions

```
float price = days * rentalPrice;
```

Group it to a single object

Car

```
String manufacturer;
String model;
int yearBuilt;
boolean isRented;
float rentalPrice;
float price = days * rentalPrice;
```

A better Way to organize…

```
String manufacturer;
String model;
int yearBuilt;
boolean isRented;
float rentalPrice;
float price = days * rentalPrice;
```

Car1

```
String manufacturer;
String model;
int yearBuilt;
boolean isRented;
float rentalPrice;
float price = days * rentalPrice;
```

Car2

```
String manufacturer;
String model;
int yearBuilt;
boolean isRented;
float rentalPrice;
float price = days * rentalPrice;
```

Car3

997 more cars ….

A better Way to organize…

Characteristics

- Manufacturer(e.g., Toyota, Tesla).
- Model (e.g., Camry, Model S).
- Year built (e.g., 2022).
- Is rented (e.g., true, false).
- Rental Price (e.g. per day, 5).

Actions

- Calculate Rental price

A Blueprint

Attributes

Methods

Class

```
String manufacturer;
String model;
int yearBuilt;
boolean isRented;
float rentalPrice;

float price (int day)
{
    return days*
rentalPrice;
}
```

What is a Class?

- Classes are central to Java and the basis for Object-Oriented Programming (OOP).
- Every Java program is essentially a class.
- Remember String and Scanner?
  - They are classes in the Java's standard library.
- A class is like a blueprint or template for creating objects.
- Defines the data (attributes/fields) and actions (methods).
- Examples: Car, Student, Book.

Classes and Objects

Class:

- Is a **type** used to declare variables.
- Defines:
  - **types of data** (**fields / Attributes**)
  - behaviours (**methods**) for objects.

Object:

- An **object** is an **instance** of a class.
- Creating an object is called **instantiation**.
- Contains **actual data** and perform **actions** (methods).

```
Car myCar = new Car();
```

Class

Variable

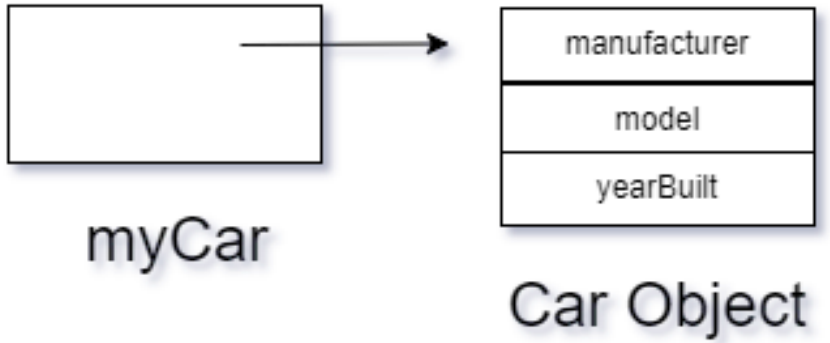
Object

Classes and Objects

Car myCar;

myCar

myCar = new Car();



myCar

Car Object

Example of Class Definition


```
public class Car {
    private String manufacturer;
    private String model;
    private int yearBuilt;
    private boolean isRented;
    private float rentalPrice;

    public float price (int days)
    {
        return days* rentalPrice;
    }
}
```

Relationship between Class and Object

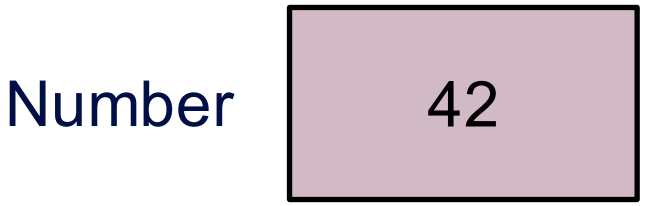
- A class defines methods that objects can perform (e.g., price() method in Car).
- All objects from a class share the **same** methods, but store **individual** data.
  - Example: myCar and yourCar both have price(), but can differ in manufacturer, model, etc.
- Methods and variable **types** belong to the class, but actual **values** belong to objects.
- Both the data items and the methods are also called **members**

Primitive vs Reference(Objects) Types

Primitive Types	Reference Types (Objects)
Single piece of data (e.g., int, char)	Multiple pieces of data and actions
Simple values	Complex entities defined by classes

Primitive vs Reference(Objects) Types

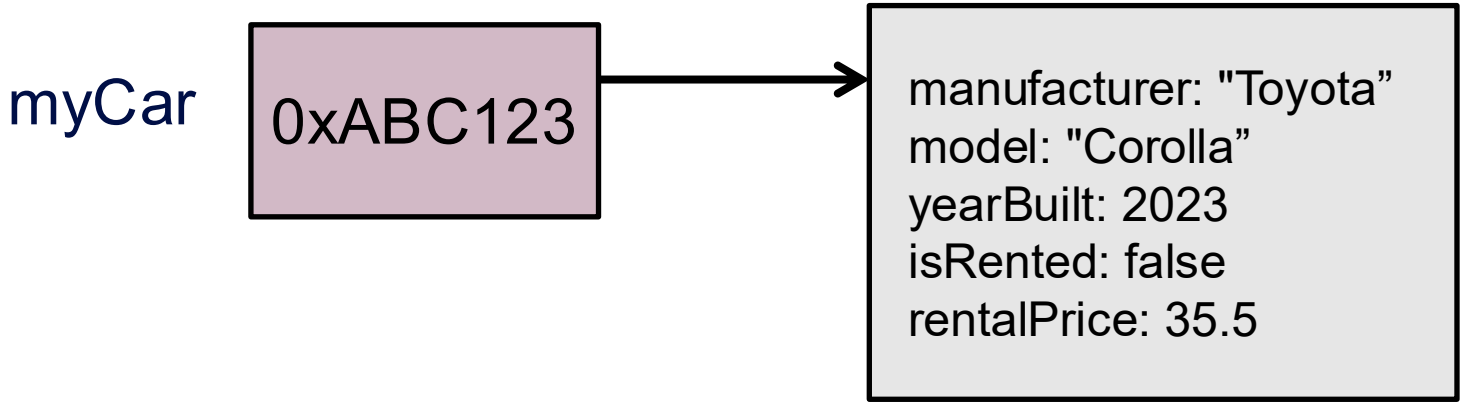
```
int number = 42 ;
```



Stores the value directly

```
Car myCar = new Car();
```

points to



Class and Method Definitions

- Classes usually defined in their own files (e.g., Car.java).
- Class definitions specify:
  - Instance variables (data items)
  - Methods (actions)
- Instance variables declaration and methods definition can be placed in any order within the class.

The *new* Operator

- An object of a class is named or declared by a variable of the class type.

  ClassName classVariable; // declaration

- The new operator must be used to create the object and associate it with its variable name:

  classVariable = new ClassName(); // instantiation

- Declaration and instantiation combined:

  ClassName classVariable = new ClassName();

Instance Variables

- Instance variables are defined within the class and can take any valid type
  - Type includes that of another class or even the same class.

```
public String manufacturer;
public int yearBuilt;
```

- The public modifier makes variables accessible from outside class.
- In order to refer to an instance variable within an object, preface it with its object name as follows:

  classVariable.instanceVariable;

- Instance variables can be both set and read as in the example:

```
Car myCar = new Car();
myCar.manufacturer = "BMW";
System.out.println("Manufacturer: " + myCar.manufacturer);
```

Methods

- Method definitions are divided into two parts: a **heading** and a **method body.**
- Syntax:

```
public void myMethod() { //heading
        //body
}
```

- Methods invoked (used) via object as follows:

```
classVariable.myMethod();
```

Types of Methods

- Void methods (no return value):

```
public void methodName(parameter_List) {
        //body
}
```

- Methods returning a value:

```
public typeReturned methodName(parameter_List) {
        //body
        return expression;
}
```

  - Return statements **terminate** method invocation.
  - Methods returning values must have a return statement:
  - Void methods may use return to exit early: → `return;`

Method Invocation

- An invocation of a method that **returns** a value can be used as an expression anywhere as follows:

```
typeReturned tRVariable;
tRVariable = objectName.methodName();
```

- Methods returning values can be used in control statements as:

```
if(objectName.methodName()) { /* ... */ }
```

- An invocation of a void method is a statement:

```
objectName.methodName();
```

Variables

- Local Variable:
  - A variable declared within a method definition
  - All method parameters

```
 public static void main(String[] args) {
        Date date1 = new Date();
....}
```

  - A variable declared within a block is local to that block.
    - When the block ends, all variables declared within the block disappear.
    - Variables declared in a loop initialization are local to it.

```
for(int i=1; i<=100; i++) { ... }
```

  - In Java, you cannot have two variables with the same name inside a single block definition

Exercise - Fix the Code

```
class Main {
    static public void main (String[] args) {
        int i = 0;
        System.out.println(i);

        if (i < 1) {
            double i = 10d;
            System.out.println(i);
        }
    }
}
```

Parameters and Arguments

Parameters:

- Methods can take additional data (parameters).
- Also called formal parameters.
- Parameter list indicates:
  - Number of parameters
  - Data types
  - Order

```
public double myMethod(int param1,
int param2, double param3) {}
```

Arguments:

- Values passed to methods at invocation:

```
int a=1, b=2, c=3;
double result = myMethod(a, b, c);
```

- Must match parameters in type and order.
- Java performs automatic type conversion if possible:
- Pass-by-Value Mechanism
  - Formal parameters initialized to argument values.
  - Primitive types passed by value; original value cannot change.
  - Objects behave differently (to be discussed later).

The *this* Keyword

- *this* is a keyword that refers to the current object
- Implicitly available in all instance methods.

```
public class Car {
    private String manufacturer;
    private int yearBuilt;
    public void writeOutput() {
        System.out.printf("%s / %s\n", manufacturer, yearBuilt);
    }
}
```

- It really means:

```
public void writeOutput() {
        System.out.printf("%s / %s\n",<the calling object>.manufacturer,
<the calling object>. yearBuilt);
    }
```

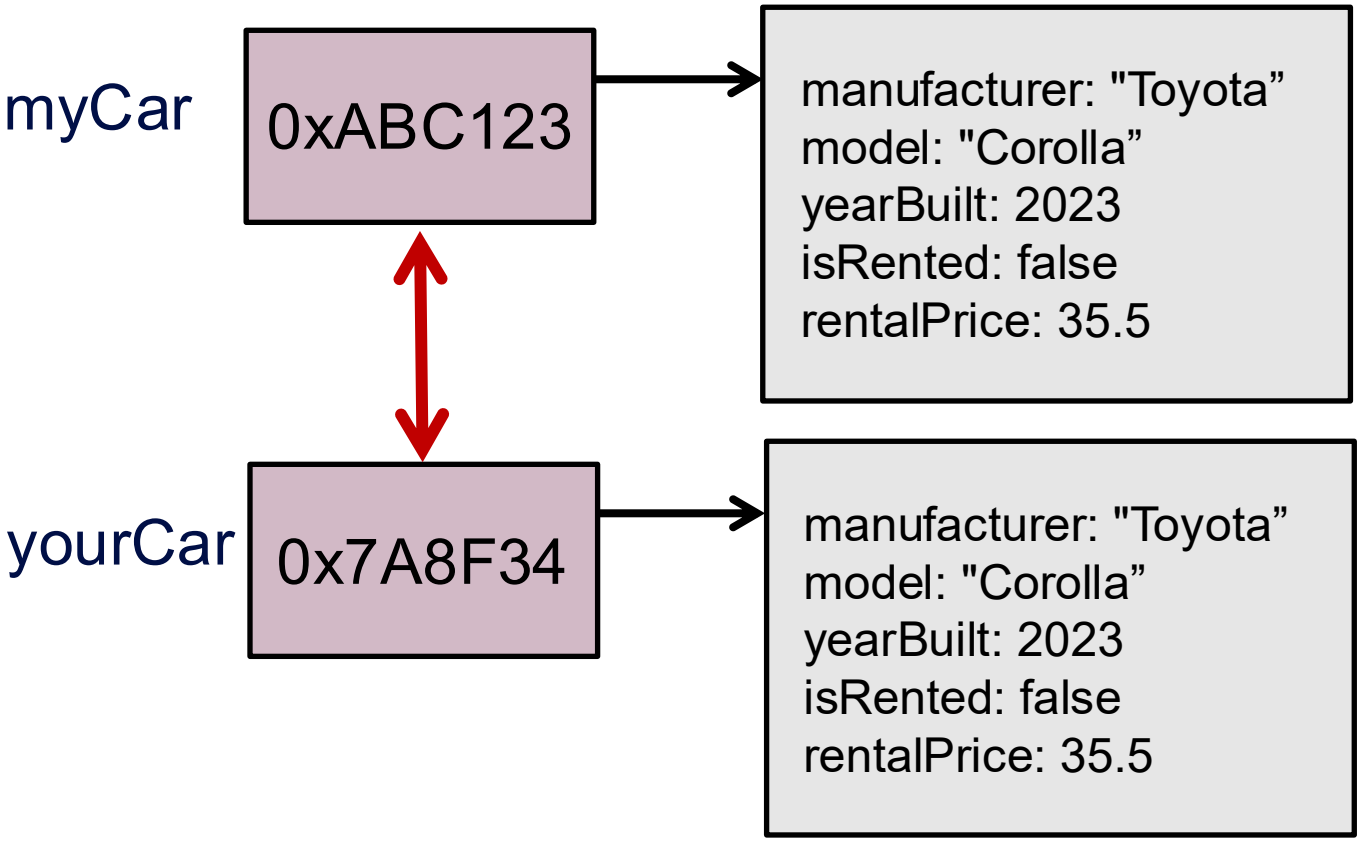
The *this* Keyword

- Explicitly using this clarifies object reference:
- Useful to distinguish between instance and local variables.

```
public class Car {
    private String manufacturer;
    private int yearBuilt;
    public void setCarDetails(String manufacturer, int yearBuilt){
        this.manufacturer = manufacturer;
     //    |_instance       |_local
        this. yearBuilt = yearBuilt;
    }
}
```

Comparing Objects in Java

- Assume we created two objects from class Car, i.e., myCar and yourCar
- Assume we assigned both objects attibutes the same values as follow:



Does
myCar == YourCar ?

NO

Using *equals()* Method

- The equals() method checks if two objects are "equal" based on their contents.

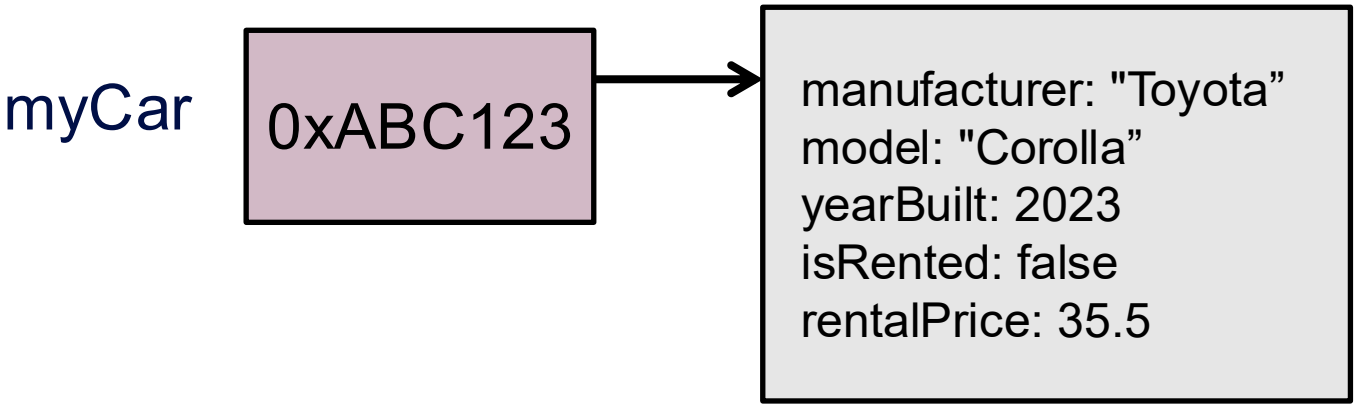
```
public class Car {
    private String manufacturer;
    private int yearBuilt;
    public boolean equals(Car anotherCar) {
       return (yearBuilt == anotherCar.yearBuilt &&
       manufacturer.equals(anotherCar.manufacturer));
     }
}
```

- Example of using equals in the main method:

```
System.out.println("my car and your car are equal: " +
myCar.equals(yourCar));
```

Printing Objects in Java

- Assume we want to print the values of the attributes of an objects from class Car, i.e., myCar.



What will be the out put of :
System.out.println(myCar);

Car@0xABC123

Using the *toString()* Method

- toString() method provides a textual representation of an object.
- Automatically called when printing objects directly.

```
public String toString() {
        return yearBuilt + " " + manufacturer;
        }
```

- So executing the print statement:

```
System.out.println(myCar);
```

- The output will be

```
2024 Toyota
```

- Java automatically calls myCar.toString().

Constructors

- A constructor is similar to methods but is not a method.
- Designed specifically to **initialize instance variables** of an object.

```
public ClassName(anyParameters) {}
```

- Must have the **same name** as the class.
- **No return** type, **not** even void.
- Typically, **overloaded**.

```
public Car(String manufacturer, int yearBuilt) {
        setCar(manufacturer, yearBuilt);
}
```

- They **differ** from methods because:
  - Cannot be invoked using the dot operator.
  - Cannot be inherited.

Constructors

- A constructor is **called** when an object of the class is created using new:

```
ClassName objectName = new ClassName(anyArgs);
```

- If a constructor is **invoked again** (using new), the first object is **discarded** and an entirely **new object** is created. → `objectName = new ClassName(anyArgs);`
- You can instead use **mutator** methods to change instance variables **after** creation.
- Constructors can **invoke**:
  - methods within their definition.
  - other constructors.
- Every constructor **implicitly** has a **this** parameter referring to the object being created, often implicit but can be explicitly used.

Types of Constructors

1- Default Constructor

- Java will create it, **Only** if **no** constructor is explicitly defined.
- has **no**-arguments
- **Initializes** variables to **default** values
- If you do not explicitly initialize instance variables, Java automatically initializes them as follows:
  - boolean types are initialized to false
  - Other primitives are initialized to the zero of their type
  - Class types and array types are initialized to null

Types of Constructors

2 - Parameterized Constructor

- Accepts arguments to set instance variables.
- If you include constructors in your class, it is good practice to always provide your own no-argument constructor.

3 - Copy Constructor

- Creates a new object by copying an existing object.

```
public Car(Car other) {
        setCar(other.manufacturer, other.yearBuilt);
}
```

Live Coding

Live Coding - Bill Example

- This is an example program that uses two classes:
  - *Bill.java*: this class holds all data and methods necessary to bill a client for a service
  - *BillingDialog.java*: this class holds the program execution code and manages the user dialogue
- Have a look at how formal parameters are used as local variables and study the program flow between the classes.

Additional Reading Resources

- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 4)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 6 and 7)
- Classes and Objects (accessible on 14-02-2024) Oracle's Java Documentation. Available at: https://docs.oracle.com/javase/tutorial/java/javaOO/index.html

The University of Melbourne (Australian University)
PRV12150/CRICOS 00116K

See you next time!

The University of Melbourne (Australian University) PRV12150/CRICOS 00116K